

Schema Patterns Playbook — Designs for High-Volume, Heavy-Throughput Workloads

A take-home reference. Every pattern lists its target workload, the canonical schema, the trap it avoids, and where it shows up in this course.

#	Pattern	Taught in	Measured in
1	Hash-Sharded Time-Series PK	Lab 3 Part D, Lab 4 Part D	Lab 8 Part D (rows/sec vs 3 other PKs)
2	Append-Only Event Log	Lab 3 Part G	Lab 14 Part B (outbox table design)
3	Per-Tenant Co-located PK	Lab 3 Part C	Lab 8 Part D , Lab 10 Part B (TPC-C uses it), Lab 15 Part B
4	Time-Bucketed Composite PK	Lab 3 review	Lab 6 (plan comparison)
5	Sharded Counter	Lab 5 Part E	Lab 8 Part E (retry rate + throughput)
6	Hot/Cold Split via Partial Index	Lab 4 Part C	Lab 15 Part B (migration redesign)
7	GLOBAL Reference Table	Lab 7 Part C	Lab 7 Part C (cross-region read latency)
8	Outbox Pattern + CDC	Lab 3 Part H	Lab 14 Part B , Lab 13 (frontier consumer)
9	TTL Table for Automatic Expiry	Lab 3 Part G	Lab 14 (outbox/idempotency tables)
10	Pre-Split + Bulk Import	Lab 3 Part E	Lab 8 Part C (range count + load time)

The anti-pattern checklist at the bottom is what to flag in a code review.

How to Prove a Pattern Is Working

A pattern you can't measure is a preference. Each pattern below carries a **Measure it** block — the specific query or command that shows the pattern doing its job. Four numbers cover most cases:

Number	Where it comes from	What it tells you
Ranges and leaseholder spread	<code>SHOW RANGES FROM TABLE t WITH DETAILS</code>	Is the write load actually distributed?
Rows/sec under concurrency	Timed concurrent writers (Lab 8 Part D)	Did the design buy throughput?
Retry / restart count	<code>crdb_internal.statement_statistics</code> → <code>maxRetries</code>	Is contention falling?
Rows read vs rows returned	<code>EXPLAIN ANALYZE</code>	Is the read path paying for the write path?

Every pattern is a trade. The `Measure it` block tells you what improved; the `Watch` line tells you what got worse. Record both before you commit to a design.

1. Hash-Sharded Time-Series PK

For: write-heavy, timestamp-ordered data — events, metrics, audit logs.

```
CREATE TABLE events (  
  account_id  UUID,  
  created     TIMESTAMPTZ DEFAULT now(),  
  id          UUID DEFAULT gen_random_uuid(),  
  payload     STRING,  
  PRIMARY KEY (account_id, created, id) USING HASH WITH (bucket_count = 16)  
);
```

Why the `id` column is in the primary key. `(account_id, created)` alone is not unique: `now()` is the **transaction** timestamp, so every row inserted by one statement gets the *same* value and the second row fails with `duplicate key value violates unique constraint`. A time-series key needs a per-row tiebreaker — a UUID here, or `clock_timestamp()` if you truly want per-row wall-clock time.

CockroachDB prepends a hidden `crdb_internal_account_id_created_shard_16` column to the key — writes scatter across 16 ranges instead of pile-up on the rightmost one.

Avoids: rightmost-range write hotspot on monotonic keys. **Watch:** `bucket_count` $\approx$ your concurrency. 16 is usually right; over 64 is rarely useful and slows range scans.

Measure it:

```
-- Distribution: the sharded table should occupy more ranges across more leaseholders  
SELECT count(*) AS ranges, count(DISTINCT lease_holder) AS leaseholders  
FROM [SHOW RANGES FROM TABLE events WITH DETAILS];  
  
-- The cost side: ordered scans now fan out across every bucket  
EXPLAIN ANALYZE SELECT * FROM events  
WHERE account_id = $1 AND created > now() - INTERVAL '1 hour'  
ORDER BY created DESC LIMIT 100;
```

Compare rows/sec against the same table with a plain `(account_id, created)` PK under 8+ concurrent writers. If the gap is small, your write rate does not yet justify the scan cost.

2. Append-Only Event Log

For: ingest-heavy logs that are never updated and aged out by time. Pair with TTL.

```
CREATE TABLE event_log (  
  id          UUID PRIMARY KEY DEFAULT gen_random_uuid(),  
  ts          TIMESTAMPTZ DEFAULT now(),  
  payload     JSONB  
) WITH (ttl_expire_after = '30 days', ttl_job_cron = '@hourly');
```

UUID PK distributes writes; TTL job sweeps expired rows automatically.

Avoids: unbounded growth and the dreaded "DELETE-as-you-go" job that competes with live traffic. **Watch:** the TTL job runs at the cron you set — too frequent and it adds load, too rare and you over-retain.

Measure it:

```
-- Is the TTL job keeping up, and what is it costing?  
SELECT job_id, status, running_status, created FROM [SHOW JOBS]  
WHERE job_type = 'ROW LEVEL TTL' ORDER BY created DESC LIMIT 5;  
  
-- Retention actually achieved  
SELECT min(ts) AS oldest_row, count(*) AS rows FROM event_log;
```

Nothing there yet? A `ROW LEVEL TTL` job only exists once the cron has fired. The **schedule** exists as soon as the table does, named `row-level-ttl: <table> [<id>]` :

```
SELECT id, label, next_run FROM [SHOW SCHEDULES] WHERE label ILIKE '%row-level-ttl%';
```

If `oldest_row` drifts past your retention window, the job is behind — lower `ttl_job_cron` or raise `ttl_delete_batch_size`, then re-check foreground write latency.

3. Per-Tenant Co-located PK

For: multi-tenant SaaS where most queries filter by tenant.

```
CREATE TABLE orders (  
  tenant_id  UUID,  
  order_id   UUID DEFAULT gen_random_uuid(),  
  customer_id UUID,  
  status     STRING,  
  total      DECIMAL(12,2),  
  PRIMARY KEY (tenant_id, order_id)  
);
```

A single tenant's data lives in a contiguous key range — queries by `tenant_id` become single-range scans. Sets up cleanly for `REGIONAL BY ROW` later.

Avoids: cross-range scans for single-tenant queries. **Watch:** one hot tenant becomes one hot range. Layer hash sharding on top if you have outliers (see #1).

Measure it:

```
-- A tenant-scoped query should be local, not distributed
EXPLAIN ANALYZE SELECT * FROM orders WHERE tenant_id = $1 ORDER BY order_id LIMIT 50;
-- want: distribution: local, and rows read ≈ rows returned

-- How the tenant's data is spread (SQL gives distribution, not traffic)
SELECT table_name, count(*) AS ranges, count(DISTINCT lease_holder) AS leaseholders
FROM [SHOW CLUSTER RANGES WITH TABLES, DETAILS]
WHERE table_name = 'orders'
GROUP BY table_name;
```

For per-range **QPS** — how you find the hot tenant — use DB Console → Advanced Debug → Hot Ranges. There is no SQL view for it: `crdb_internal.ranges_no_leases` carries neither `table_name` nor `lease_holder`, and `crdb_internal.cluster_replicas` does not exist.

4. Time-Bucketed Composite PK

For: per-tenant time-range queries that need ordering.

```
CREATE TABLE metrics (
  tenant_id    UUID,
  hour_bucket  TIMESTAMPTZ,          -- truncated to the hour
  metric_id    UUID DEFAULT gen_random_uuid(),
  value        DECIMAL,
  PRIMARY KEY (tenant_id, hour_bucket, metric_id)
);

-- Insert:
INSERT INTO metrics (tenant_id, hour_bucket, value)
VALUES ('...', date_trunc('hour', now()), 42.0);
```

The bucket column lets the optimizer prune to one hour's worth of keys without an extra index.

Avoids: unbounded range scans; sequential keys without coarser-grained pruning.

5. Sharded Counter

For: very-high-frequency counters (page views, likes, gauges).

```

CREATE TABLE counter_shards (
  name  STRING,
  shard INT,
  n     INT DEFAULT 0,
  PRIMARY KEY (name, shard)
);

-- Pre-create shards
INSERT INTO counter_shards (name, shard)
SELECT 'page_views', g FROM generate_series(0, 15) g;

-- Increment: the APPLICATION picks the shard and passes it as a parameter
UPDATE counter_shards SET n = n + 1
WHERE name = 'page_views' AND shard = $1;      -- $1 = random.randint(0, 15)

-- Read: sum across shards
SELECT sum(n) FROM counter_shards WHERE name = 'page_views';

```

Do not put `random()` in the predicate. `WHERE shard = (random()*16)::INT` looks equivalent and is not: `random()` is **volatile and evaluated per row scanned**, so the statement matches a random *number* of rows — sometimes zero (the increment is silently lost), sometimes several (it double-counts). Measured over 200 sequential increments on an idle cluster, the predicate form landed 193 and a scalar-subquery form landed 211; the client-chosen shard landed exactly 200. Pick the shard where you have a stable value: in the application.

Avoids: SQLSTATE 40001 retry storms on a single-row counter. **Watch:** reads are now $O(\text{shards})$. Don't shard a low-frequency counter — pay the read cost for nothing.

Measure it:

```

-- The number that justifies the pattern: retries before vs after
SELECT substring(metadata->'query', 1, 60) AS stmt,
  (statistics->'statistics'->'cnt')::INT AS executions,
  (statistics->'statistics'->'maxRetries')::INT AS max_retries,
  round((statistics->'statistics'->'svclat'->'mean')::FLOAT * 1000, 2) AS mean_ms
FROM crdb_internal.statement_statistics
WHERE metadata->'query' LIKE '%counter%'
ORDER BY executions DESC;

```

Run 16 concurrent bumpers against both designs (Lab 8 Part E). The single-row version's retry count grows with concurrency; the sharded version's stays near zero. **Shard count:** start at your peak concurrent writer count, round to a power of two, cap around 64.

6. Hot/Cold Split via Partial Index

For: big table where 99% of queries hit a small recent or active slice.

```
CREATE INDEX orders_open
ON orders(total DESC)
STORING (customer_id)
WHERE status = 'open';
```

The partial index holds only "open" rows — sorted by `total`, with the columns the hot query needs. Updates that change `status` away from `open` remove the row from the index in the same transaction.

Avoids: wasting index space on rows that are never queried; slow `ORDER BY` over the whole table.

Measure it:

```
-- Is the partial index actually being used?
SELECT ti.descriptor_name AS table_name, ti.index_name,
       s.total_reads, s.last_read
FROM crdb_internal.index_usage_statistics s
JOIN crdb_internal.table_indexes ti
  ON s.table_id = ti.descriptor_id AND s.index_id = ti.index_id
WHERE ti.descriptor_name = 'orders';
```

An index with `total_reads = 0` after a representative workload is pure write amplification — drop it. This query is also how you audit "we indexed everything just in case".

7. GLOBAL Reference Table

For: small read-anywhere lookup data — country codes, currencies, feature flags.

```
CREATE TABLE country_codes (
  code STRING(2) PRIMARY KEY,
  name STRING NOT NULL
) LOCALITY GLOBAL;
```

Non-blocking transactions make this look like it has a leaseholder in every region — reads are fast everywhere.

Avoids: cross-region reads on hot lookup paths. **Watch:** writes are slow (cross-region clock skew waits). Don't `LOCALITY GLOBAL` a write-heavy table.

Measure it: time the same `SELECT` from a gateway in each region — a `GLOBAL` table should be fast everywhere. Then time an `UPDATE` and confirm you can live with it. If the write latency is unacceptable, the table is not reference data and does not belong here.

8. Outbox Pattern + CDC

For: publishing events atomically with business writes. Replaces "write to DB and Kafka from app code" which can dual-write inconsistently.

```
-- Same transaction as the business write:
BEGIN;
INSERT INTO orders (...) VALUES (...);
INSERT INTO events_outbox (topic, payload)
  VALUES ('orders.created', jsonb_build_object('id', $1, ...));
COMMIT;

-- One-time setup:
CREATE TABLE events_outbox (
  id        UUID PRIMARY KEY DEFAULT gen_random_uuid(),
  topic     STRING NOT NULL,
  payload   JSONB,
  created   TIMESTAMPTZ DEFAULT now()
) WITH (ttl_expire_after = '7 days');

CREATE CHANGFEED FOR TABLE events_outbox
  INTO 'kafka://...'
  WITH updated, resolved = '10s';
```

Avoids: the dual-write inconsistency where you write to the DB, the process dies, and Kafka never gets the event (or vice versa). **Watch:** events are at-least-once. Downstream consumers must be idempotent.

Measure it:

```
-- Atomicity: after a simulated crash before COMMIT, both counts must be unchanged
SELECT (SELECT count(*) FROM orders) AS orders, (SELECT count(*) FROM events_outbox) AS
  events;

-- Delivery lag, for alerting (Lab 9, Lab 13)
SELECT job_id, (now() - hlc_to_timestamp(high_water_timestamp)) AS lag
FROM [SHOW CHANGFEED JOBS] WHERE status = 'running';
```

Design rules that matter more than the code: UUID primary key (append-only at high rate), row-level TTL (it is a buffer, not a log), and **no status column** — a polled `WHERE status='pending'` queue serializes every worker on one range.

9. TTL Table for Automatic Expiry

For: rows with a natural expiration — sessions, tokens, soft locks, short-lived caches.

```
-- TTL by static duration:
CREATE TABLE sessions (
  id          UUID PRIMARY KEY DEFAULT gen_random_uuid(),
  user_id     UUID,
  data        JSONB
) WITH (ttl_expire_after = '24 hours', ttl_job_cron = '@hourly');

-- TTL driven by a per-row expression:
CREATE TABLE access_tokens (
  id          UUID PRIMARY KEY,
  expires     TIMESTAMPTZ
) WITH (ttl_expiration_expression = 'expires', ttl_job_cron = '*/10 * * * *');
```

Background TTL job sweeps expired rows on schedule.

Avoids: application-managed cleanup; `DELETE FROM ... WHERE created < now() - INTERVAL ...` that competes with live traffic. **Watch:** the TTL job is a job — visible in `[SHOW JOBS]`, pausable, monitorable. Tune `ttl_job_cron` to your retention SLO.

10. Pre-Split + Bulk Import

For: one-time loads of sequential-key data — analytics imports, archive restores, migrations.

```
ALTER TABLE big_import SPLIT AT
  VALUES (100000), (200000), (300000), (400000), (500000),
          (600000), (700000), (800000), (900000);

IMPORT INTO big_import (id, payload)
  CSV DATA ('s3://my-bucket/data.csv?...');
```

Splits the table into N empty ranges before the load — every range gets writes from the start.

Avoids: writing 100M rows into a single range and waiting for the splitter to catch up.

Measure it:

```
SELECT count(*) AS ranges FROM [SHOW RANGES FROM TABLE big_import];           -- before the load
SELECT count(DISTINCT lease_holder) FROM [SHOW RANGES FROM TABLE big_import WITH DETAILS];
```

Load the same data into a split and an un-split copy and compare elapsed time (Lab 8 Part C). **Sizing the splits:** one per ~512 MiB of expected data, or one per node × 3–10, whichever is larger.

Then release them — manual splits are pinned until you do, and ranges can never merge back:

```
ALTER TABLE big_import UNSPLIT ALL;
```

Anti-Patterns to Spot in Code Review

Pattern	Why it hurts	Replace with
<code>id SERIAL PRIMARY KEY</code> on a high-volume table	Rightmost-range hotspot	UUID PK (#1) or hash-sharded composite (#1)
<code>UPDATE counters SET n = n + 1 WHERE id = '...'</code>	40001 retry storm	Sharded Counter (#5)
Low-cardinality lead column in PK (<code>region</code> , <code>status</code>)	One range per value → severe hotspot	Reorder to put a high-cardinality column first
Indexes on every column "just in case"	Write amplification multiplies inserts	Audit with <code>crdb_internal.index_usage_statistics</code> ; drop unused
<code>DELETE FROM logs WHERE created < ...</code> as a recurring job	Competes with live traffic, locks	TTL Table (#9)
Application code that writes to Kafka <i>and</i> the database	Dual-write inconsistency	Outbox Pattern + CDC (#8)
Reading then writing without <code>BEGIN ... COMMIT</code>	No atomicity guarantees	Wrap in an explicit transaction
<code>SELECT *</code> everywhere	Forces PK lookup; defeats covering indexes	List columns; add <code>STORING</code> for hot paths
One big bulk load into an un-split table	Initial single-range write storm	Pre-Split + Bulk Import (#10)
<code>LOCALITY GLOBAL</code> on a write-heavy table	Cross-region clock-skew waits on every commit	Use <code>REGIONAL BY ROW</code> or <code>REGIONAL BY TABLE IN</code> instead

A Quick Decision Tree

When designing a new table, ask in this order:

1. What's the write pattern?

- One-shot bulk import → Pattern #10 (Pre-Split + IMPORT INTO)
- Steady high-throughput inserts, timestamp-ordered → #1 (Hash-Sharded Time-Series PK)
- Many writes per tenant, mostly read by tenant → #3 (Per-Tenant Co-located PK)
- Bursts to a single row (counter, gauge) → #5 (Sharded Counter)
- Application produces events for downstream → #8 (Outbox + CDC)

2. What's the lifecycle?

- Rows expire naturally → #9 (TTL Table)
- Append-only logs → #2 (Append-Only Event Log = #9 + UUID PK)
- Long-lived but rarely-updated reference data → #7 (GLOBAL) if multi-region

3. What's the read pattern?

- Most queries hit a hot subset → #6 (Hot/Cold Split via Partial Index)
- Time-range queries per tenant → #4 (Time-Bucketed Composite PK)
- Read from every region → #7 (GLOBAL) for small tables; per-row regional for large

Mix and match — production schemas usually combine 2-4 of these patterns. The anti-pattern checklist catches the rest.

The Cost Side: What Every Pattern Charges You

Schema decisions have a hardware price. At replication factor 3, one logical insert into a table with two secondary indexes is:

1 row × 3 KV writes (base + 2 indexes) × 3 replicas = 9 physical writes

Adding one index to a hot table raises your write hardware requirement by roughly a third. That is why the sizing exercise in Lab 10 takes the index count as an *input*: the schema decides the bill.

Decision	Write cost	Read benefit	Measure with
Add a secondary index	+1 KV write × RF per row	Avoided scan	Lab 8 challenge 2 (rows/sec vs index count)
Hash-shard a PK	~none	Ordered scans fan out	Lab 8 Part D
Shard a counter	~none	Reads become O(shards)	Lab 8 Part E
Co-locate by tenant	~none	Single-range tenant queries	EXPLAIN ANALYZE distribution
LOCALITY GLOBAL	Cross-region commit waits	Local reads everywhere	Lab 7 Part C
Row-level TTL	Background job load	No cleanup job of your own	SHOW JOBS + foreground p99

Using This Playbook in a Design Review

1. **Name the write pattern** — bulk, steady, bursty-to-one-row, or event-producing.
2. **Name the read pattern** — by tenant, by time range, by hot subset, from every region.
3. **Pick the pattern** from the decision tree above.
4. **State the trade** — every pattern's `Watch` line is a cost someone will pay.
5. **Bring a number** — ranges, rows/sec, retries, or rows-read. Lab 8 shows how to get each one in under ten minutes on a laptop.

A design review that ends without a number is a preference, and preferences do not survive production traffic.